

Classification Models, Evaluation Metrics & Handling Imbalanced Data

Submitted by: AI & Machine Learning Intern **Task:** Task 4 — Classification Model Evaluation **Dataset:** Breast Cancer Wisconsin (Diagnostic) Dataset — scikit-learn

1. Introduction

Machine Learning Classification is a supervised learning approach in which a model learns from labeled training data to assign new, unseen observations to one of a predefined set of categories, or classes. Instead of producing a continuous numerical output, a classification model produces a discrete label — for example, predicting whether a tumour is "malignant" or "benign," whether an email is "spam" or "not spam," or whether a transaction is "fraudulent" or "legitimate."

Regression vs. Classification. The fundamental difference between regression and classification lies in the nature of the target variable. Regression predicts a continuous numeric value, such as house price, temperature, or sales revenue, where the output can take any value within a range. Classification, on the other hand, predicts a categorical label drawn from a finite set of classes. While regression models are evaluated using metrics such as Mean Squared Error or R^2 , classification models are evaluated using metrics such as accuracy, precision, recall, and the area under the ROC curve — all of which are discussed in detail later in this report.

Importance of Classification in Real-World Applications. Classification underpins many of the most impactful applications of machine learning:

- **Medical Diagnosis:** Classification models assist clinicians by predicting whether a patient's test results indicate a disease (e.g., cancer, diabetes), enabling earlier detection and treatment.
- **Spam Detection:** Email providers use classification to automatically separate spam from legitimate messages, protecting users from phishing and unwanted content.
- **Fraud Detection:** Banks and payment processors classify transactions as fraudulent or legitimate in real time, reducing financial losses.
- **Customer Churn Prediction:** Businesses classify customers as likely to churn or stay, allowing proactive retention strategies.

This report works through a complete classification pipeline — from data preparation to model evaluation — using the Breast Cancer Wisconsin dataset, and discusses how evaluation metrics beyond accuracy provide a more trustworthy picture of model performance, especially when class imbalance is present.

2. Dataset Description

The notebook uses the **Breast Cancer Wisconsin (Diagnostic) Dataset**, loaded directly from scikit-learn's built-in datasets module (`load_breast_cancer`). This is a widely used benchmark dataset for binary classification tasks in medical machine learning.

Property	Value
Dataset name	Breast Cancer Wisconsin (Diagnostic)
Number of samples	569
Number of features	30 (all numeric, derived from cell nuclei measurements)
Target classes	2 — Malignant (0), Benign (1)
Class distribution	Malignant: 212 samples (~37.3%) · Benign: 357 samples (~62.7%)

The 30 features are computed from digitized images of fine needle aspirate (FNA) biopsies and describe characteristics of cell nuclei, such as radius, texture, perimeter, area, smoothness, compactness, concavity, symmetry, and fractal dimension — each reported as a mean, standard error, and "worst" (largest) value.

Why this dataset is suitable for binary classification: The target variable has exactly two mutually exclusive outcomes — malignant or benign — which is the defining characteristic of a binary classification problem. The dataset also exhibits a moderate, realistic class imbalance (roughly 37% vs. 63%), making it a useful case study for examining how imbalance affects model evaluation, while still containing enough samples in the minority class for a model to learn meaningful patterns.

3. Data Preprocessing

Before training, the notebook applies a standard three-step preprocessing pipeline:

Train-Test Split. The dataset was divided into a training set and a testing set using an 80:20 split (`test_size=0.2`), resulting in 455 training samples and 114 testing samples. The split was performed with a fixed `random_state=42` to ensure reproducibility.

Stratification. The split used `stratify=y`, which ensures that the proportion of malignant and benign cases in the training and testing sets closely mirrors the proportion in the full dataset. This is particularly important for an imbalanced dataset such as this one, because a non-stratified random split could, by chance, under-represent the minority class (malignant) in either the training or testing set, leading to a model that is trained on or evaluated against an unrepresentative sample. With stratification, the test set contains 42 malignant and 72 benign cases — proportionally consistent with the overall 37:63 split.

Feature Scaling using StandardScaler. The 30 input features were standardized using `StandardScaler`, which transforms each feature to have zero mean and unit variance. The scaler was fit only on the training data (`fit_transform`) and then applied to the test data using the parameters learned from training (`transform`), which prevents information from the test set leaking into the training process.

Why scaling was required: The features in this dataset vary enormously in scale — for example, "mean area" can be in the hundreds or thousands, while "mean smoothness" is a small decimal value. Logistic Regression estimates coefficients using a distance- and gradient-based optimization process, and features with larger numeric ranges would disproportionately dominate the loss function and gradient updates if left unscaled. Standardization places all features on a comparable scale, which leads to faster, more stable convergence and prevents any single feature from artificially dominating the model purely because of its units of measurement.

4. Logistic Regression Model

Working principle. Logistic Regression is a linear model adapted for classification. It computes a weighted linear combination of the input features and passes this value through the sigmoid (logistic) function, which maps any real number into a probability between 0 and 1. A threshold (typically 0.5) is then applied to this probability to assign a discrete class label. Despite the name "regression," it is fundamentally a classification algorithm.

Why Logistic Regression was selected as a baseline. Logistic Regression was chosen as the starting model because it is computationally efficient, easy to interpret (its coefficients indicate the direction and strength of each feature's influence on the outcome), and performs well when classes are reasonably linearly separable — which is often true for clinical/biological measurement data like this dataset. It provides a strong, simple reference point against which more complex models can be compared.

Training procedure used in the notebook. The model was instantiated as `LogisticRegression(max_iter=1000)`, with the increased iteration limit ensuring the optimizer fully converges on the standardized feature set. It was trained using `model.fit(X_train_scaled, y_train)` on the 455 scaled training samples, and predictions were then generated on the scaled test set using `model.predict(X_test_scaled)`.

5. Model Evaluation

5.1 Confusion Matrix

The confusion matrix produced by the notebook for the baseline Logistic Regression model on the 114-sample test set is:

	Predicted Malignant (0)	Predicted Benign (1)
Actual Malignant (0)	41 (True Negative*)	1 (False Positive)
Actual Benign (1)	1 (False Negative)	71 (True Positive*)

Note: in this report, "Benign" (class 1) is treated as the positive class, consistent with scikit-learn's default convention of treating the higher-numbered label as positive.

- **True Positives (TP) = 71:** Cases correctly predicted as benign when they were actually benign.
- **True Negatives (TN) = 41:** Cases correctly predicted as malignant when they were actually malignant.
- **False Positives (FP) = 1:** One case predicted as benign that was actually malignant — the most clinically serious type of error here, since a malignant tumour would be missed.
- **False Negatives (FN) = 1:** One case predicted as malignant that was actually benign — a "false alarm" that would likely trigger unnecessary further testing.

Interpretation: Out of 114 test cases, the model misclassified only 2 — one in each direction. This is an exceptionally strong result, indicating the model correctly separates the two classes for all but two borderline samples. The single false positive (a missed malignant case) is the more clinically critical error and would be the first thing flagged for review in a real medical deployment, even though it represents a very small fraction of the test set.

5.2 Classification Report

The exact classification report generated by the notebook for the baseline Logistic Regression model is:

Class	Precision	Recall	F1-Score	Support
0 (Malignant)	0.98	0.98	0.98	42
1 (Benign)	0.99	0.99	0.99	72
Accuracy			0.98	114
Macro Avg	0.98	0.98	0.98	114
Weighted Avg	0.98	0.98	0.98	114

Interpretation of each metric:

- **Precision** answers: "Of all the cases the model predicted as a given class, how many were correct?" A precision of 0.99 for benign means that when the model predicts

"benign," it is correct 99% of the time.

- **Recall** answers: "Of all the actual cases of a given class, how many did the model correctly identify?" A recall of 0.98 for malignant means the model correctly caught 98% of the truly malignant cases — missing only 1 out of 42.
- **F1-Score** is the harmonic mean of precision and recall, providing a single balanced measure that penalizes models which sacrifice one metric heavily for the other. An F1-score of 0.98–0.99 here indicates precision and recall are both high and well balanced.
- **Accuracy** is the overall proportion of correct predictions across both classes — 98% in this case, meaning 112 out of 114 test samples were classified correctly.

5.3 Why Accuracy Alone Is Insufficient

Although the baseline model achieves a high accuracy of 98%, accuracy alone can be a misleading indicator of performance, particularly on imbalanced datasets. Consider a hypothetical (not this dataset's) scenario where 95% of patients in a test set do not have a disease and only 5% do — a model that simply predicts "no disease" for every single patient would achieve 95% accuracy while having a recall of 0% for the disease class, making it clinically useless and potentially dangerous, since it would fail to flag a single true case.

This is precisely why precision, recall, and F1-score are reported alongside accuracy: they reveal how the model performs *per class*, not just in aggregate. In the current dataset, the moderate imbalance (37% malignant vs. 63% benign) makes it important to check that the minority class (malignant) is not being under-served — and indeed, the per-class breakdown confirms strong recall (0.98) for the malignant class as well, validating that the high accuracy is not an artifact of the model defaulting toward the majority class.

6. ROC Curve and AUC Analysis

The notebook computes the Receiver Operating Characteristic (ROC) curve and the corresponding Area Under the Curve (AUC) score using the predicted probabilities from the Logistic Regression model.

AUC Score (from notebook output): 0.9954 (approximately 0.995, displayed as 1.00 in the legend of the plotted curve due to rounding)

Key concepts:

- **False Positive Rate (FPR):** The proportion of actual negative cases (malignant) that were incorrectly classified as positive (benign). It is calculated as $FP / (FP + TN)$.
- **True Positive Rate (TPR), also called Recall or Sensitivity:** The proportion of actual positive cases (benign) that were correctly identified. It is calculated as $TP / (TP + FN)$.
- **ROC Curve:** A plot of TPR (y-axis) against FPR (x-axis) across all possible classification thresholds. It illustrates the trade-off between correctly identifying positive cases and incorrectly flagging negative cases as the decision threshold is varied.

- **AUC Interpretation:** The AUC represents the probability that the model ranks a randomly chosen positive case higher than a randomly chosen negative case. An AUC of 0.5 indicates performance no better than random guessing, while an AUC of 1.0 indicates perfect separation between classes.

The ROC curve generated in the notebook is reproduced below:

[ROC Curve Figure — see attached image]

Discussion: With an AUC of 0.9954, the model demonstrates near-perfect class separation. This means that, almost without exception, the model assigns a higher predicted probability of "benign" to actual benign cases than to actual malignant cases. Combined with the confusion matrix and classification report, this confirms that the baseline Logistic Regression model is a strong classifier for this dataset, with both excellent threshold-based performance (98% accuracy) and excellent probability-ranking performance (AUC $\approx$ 0.995).

7. Handling Class Imbalance

What is class imbalance? Class imbalance occurs when the classes in a dataset are not represented in roughly equal proportions — one class (the majority class) significantly outnumbers the other (the minority class). In this dataset, the imbalance is moderate: benign cases (357, $\sim$ 63%) outnumber malignant cases (212, $\sim$ 37%).

Why it is a problem in machine learning: Many classification algorithms are trained to minimize overall error, which can be achieved by simply favoring the majority class. In datasets with severe imbalance, this can result in a model that performs very well on the majority class but very poorly on the minority class — which is frequently the class of greatest practical interest (e.g., detecting the disease, the fraud, or the churned customer).

Why accuracy can become misleading: As discussed in Section 5.3, a model can achieve high accuracy by simply predicting the majority class most or all of the time, while failing to capture any of the minority class. This is why precision, recall, and F1-score — particularly for the minority class — must be examined alongside accuracy whenever a dataset shows imbalance.

Implementation of `class_weight="balanced"`: To address this, the notebook trains a second Logistic Regression model using the parameter `class_weight="balanced"`:

```
python

model_balanced = LogisticRegression(class_weight="balanced", max_iter=1000)
model_balanced.fit(X_train_scaled, y_train)
```

This parameter automatically adjusts the weight assigned to each class during model training, inversely proportional to class frequency. In effect, errors made on the minority class (malignant) are penalized more heavily than errors on the majority class (benign)

during optimization. This discourages the model from "ignoring" the minority class in pursuit of lower overall error and instead pushes the decision boundary toward more balanced sensitivity across both classes.

Comparison against baseline model: The notebook fits `model_balanced` on the training data but does not generate predictions or a classification report for it. Therefore, no evaluation metrics (accuracy, precision, recall, F1, AUC) are available in the notebook output for the balanced model, and this report does not fabricate or estimate such figures. Based on the underlying mechanism, the expected effect of `class_weight="balanced"` would be to potentially increase recall on the malignant (minority) class — possibly at a small cost to precision or accuracy on the benign class — since the model is explicitly penalized more for missing malignant cases. Given that the baseline model already achieves a malignant-class recall of 0.98 (41/42 correct) on this dataset, there is very limited room for the balanced model to materially improve recall further. A direct quantitative comparison would require generating predictions from `model_balanced` on the test set, which is recommended as a follow-up step.

8. Decision Tree Classifier Comparison

Working principle. A Decision Tree Classifier builds a tree-like structure of sequential if-else decision rules based on feature values, recursively splitting the dataset to maximize the separation between classes at each node (commonly using criteria such as Gini impurity or information gain). Each path from the root to a leaf represents a sequence of decisions that ultimately assigns a class label.

Model training process. In the notebook, a `DecisionTreeClassifier(random_state=42)` was trained directly on the *unscaled* training data (`X_train`, `y_train`) — this is appropriate because, unlike Logistic Regression, decision trees split on feature thresholds rather than computing distances or gradients, so they are not sensitive to feature scale. Predictions were then generated on the unscaled test set (`X_test`).

Actual results from notebook (Decision Tree):

Class	Precision	Recall	F1-Score	Support
0 (Malignant)	0.85	0.93	0.89	42
1 (Benign)	0.96	0.90	0.93	72
Accuracy			0.91	114
Macro Avg	0.90	0.92	0.91	114
Weighted Avg	0.92	0.91	0.91	114

Model Comparison Table

Metric	Logistic Regression (Baseline)	Decision Tree
Accuracy	0.98	0.91
Precision (Malignant)	0.98	0.85
Precision (Benign)	0.99	0.96
Recall (Malignant)	0.98	0.93
Recall (Benign)	0.99	0.90
F1-Score (Malignant)	0.98	0.89
F1-Score (Benign)	0.99	0.93
Macro F1-Score	0.98	0.91
AUC Score	0.9954	Not computed in notebook

Discussion:

- **Which model performed better?** Logistic Regression clearly outperforms the Decision Tree on every reported metric in this notebook — accuracy (0.98 vs. 0.91), precision, recall, and F1-score across both classes. The performance gap is most visible in malignant-class precision (0.98 vs. 0.85), meaning the Decision Tree produced more false positives for the malignant class relative to Logistic Regression.
- **Which model is easier to interpret?** The Decision Tree is generally considered more directly interpretable, since its decision path can be visualized and followed as a sequence of human-readable if-else rules on specific feature thresholds. Logistic Regression is also interpretable, but through coefficient magnitudes and signs rather than explicit decision rules — both are reasonably transparent compared to "black-box" models like deep neural networks.
- **Which model is more prone to overfitting?** Decision Trees, particularly when grown without depth restriction or pruning (as in this notebook, where no `max_depth` was set), are well known to be more prone to overfitting the training data, since they can keep splitting until nodes are pure. Logistic Regression, being a simpler linear model, has lower variance and is generally less prone to overfitting, especially on a dataset of this size (455 training samples vs. 30 features).
- **Which model would be preferred for deployment?** Based on the results obtained in this notebook, Logistic Regression would be preferred for deployment. It achieves higher accuracy, precision, recall, and F1-score across both classes, and its near-perfect AUC score (0.9954) further confirms strong, reliable probability-based

ranking — an important property for a clinical decision-support context where a probability threshold might need to be tuned.

9. Final Model Selection

Based on the complete set of evaluation results obtained in this notebook, the **baseline Logistic Regression model is selected as the final recommended model**, justified as follows:

Criterion	Logistic Regression	Decision Tree	Better Model
Accuracy	0.98	0.91	Logistic Regression
Precision	0.98-0.99	0.85-0.96	Logistic Regression
Recall	0.98-0.99	0.90-0.93	Logistic Regression
F1-Score	0.98-0.99	0.89-0.93	Logistic Regression
AUC Score	0.9954	Not available	Logistic Regression
Generalization ability	Lower overfitting risk (linear, regularized)	Higher overfitting risk (unpruned tree)	Logistic Regression

Logistic Regression outperforms the Decision Tree on every metric that was computed in the notebook, and its theoretical generalization properties (as a simpler, lower-variance linear model with no signs of unconstrained depth growth) further support its selection over the unpruned Decision Tree. The near-perfect AUC score of 0.9954 additionally confirms that the model's predicted probabilities are highly reliable for ranking and threshold-based decision-making, which is valuable in a real-world diagnostic support setting.

It is worth noting that the `class_weight="balanced"` variant of Logistic Regression was trained in the notebook but not evaluated with predictions or metrics; were this report to extend the analysis, generating and evaluating its predictions would be a natural next step before finalizing a production model — particularly to confirm whether it offers any further improvement in malignant-class recall over the already-strong baseline.

10. Conclusion

This task involved building and evaluating classification models on the Breast Cancer Wisconsin dataset to predict whether a tumour is malignant or benign.

Objectives achieved:

- Performed a stratified train-test split (80:20) and applied feature scaling using `StandardScaler`.
- Trained a baseline Logistic Regression model and evaluated it using a confusion matrix, full classification report, and ROC-AUC analysis.
- Trained a second Logistic Regression model with `class_weight="balanced"` to address class imbalance.
- Trained a Decision Tree Classifier as a comparison model and evaluated it with a classification report.

Key findings:

- The baseline Logistic Regression model achieved a test accuracy of **98%**, with precision and recall both at 0.98–0.99 across classes, misclassifying only 2 of 114 test samples (1 false positive, 1 false negative).
- The ROC-AUC score of **0.9954** confirms near-perfect class separability and ranking quality.
- The Decision Tree Classifier achieved a lower accuracy of **91%**, with a more noticeable drop in malignant-class precision (0.85), indicating it is the weaker of the two models on this dataset.
- Accuracy alone was shown to be an insufficient evaluation criterion in classification problems, particularly under class imbalance, reinforcing the need for precision, recall, F1-score, and AUC as complementary diagnostics.

Evaluation insights: The exercise highlights that a simple, well-regularized linear model (Logistic Regression) can outperform a more flexible but unconstrained model (Decision Tree) on a dataset of this size and structure, and that class imbalance handling techniques such as `class_weight="balanced"` provide a mechanism to explicitly prioritize minority-class performance — though their benefit must always be confirmed through evaluation on held-out data rather than assumed.

Final recommended model: The baseline **Logistic Regression model** (trained on standardized features, without class weighting) is recommended as the final model, on the strength of its accuracy, balanced precision/recall, and near-perfect AUC score observed in this notebook.

End of Report.